NLP Practicals - Code Cheat Sheet

#1A Sentence & word tokenize

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
text = "Sherlock Holmes is the best Detective ever."
sentence = sent_tokenize(text)
words = word_tokenize(text)
print(sentence)
for i, sent in enumerate(sentence, 1):
    print(f"{i}, {sent}")
print(f"\nTotal tokens:{len(words)}")
print(f"The first 10 tokens: {words[:10]}")
```

#1B Stopword removal

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
nltk.download('punkt'); nltk.download('stopwords')
stopword_set = set(stopwords.words('english'))
text = input('Enter a sentence: ')
tokens = word_tokenize(text)
print("Count of tokens before:", len(tokens))
filtered_tokens = [t.lower() for t in tokens
                   if t.lower() not in stopword_set]
print("Count of tokens after:", len(filtered_tokens))
```

#1C Porter stemmer

```
import nltk
from nltk.stem import PorterStemmer
nltk.download('punkt')
ps = PorterStemmer()
text = input("Enter a sentence: ")
tokens = nltk.word_tokenize(text)
stemmed_words = [ps.stem(t) for t in tokens]
print("Original tokens:", tokens)
print("Stemmed tokens:", stemmed_words)
```

#2A POS tagging

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
sentence = "The quick brown fox jumps over the lazy dog."
tokens = word_tokenize(sentence)
tagged_output = pos_tag(tokens)
print("Input Sentence:", sentence)
print("\nPOS Tagged Output:")
for word, tag in tagged_output:
    print(f"{word}: {tag}")
```

#2B Lemmatization

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
input_text = "The cats were chasing mice through the yard."
tokens = word_tokenize(input_text)
lemmatized_tokens = [lemmatizer.lemmatize(t) for t in tokens]
lemmatized_text = " ".join(lemmatized_tokens)
print("Original Text:  ", input_text)
print("Lemmatized Text:", lemmatized_text)
```

#3A Chunking (RegexpParser)

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
from nltk.chunk import RegexpParser
sentence = "The smart student solved difficult problem in the classroom"
tokens = word_tokenize(sentence)
tagged_words = pos_tag(tokens)
grammar = r"""
  NP : {<DT>?<JJ>*<NN.*>+}
  PP : {<IN><NP>}
  VP : {<VB.*><NP|PP>*}
  CLAUSE : {<NP><VP>}
"""
parser = RegexpParser(grammar)
chunk_tree = parser.parse(tagged_words)
print("Chunk Tree Output : ")
print(chunk_tree)
chunk_tree.draw()
```

#3B Named Entity Recognition (ne_chunk)

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
from nltk.chunk import ne_chunk
input_text = "Barack Obama visited Google headquarters in California."
tokens = word_tokenize(input_text)
tagged_tokens = pos_tag(tokens)
ner_tree = ne_chunk(tagged_tokens)
print("Named Entity Recognition Output : ")
print(ner_tree)
ner_tree.draw()
```

#4A TF-IDF

```
import nltk, string
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
nltk.download('punkt'); nltk.download('stopwords')
corpus = [
    "The quick brown fox jumps over the lazy dog.",
    "A brown fox is fast and quick.",
    "The lazy dog sleeps."
]
def preprocess_text(doc):
    tokens = nltk.word_tokenize(doc.lower())
    stop_words = set(stopwords.words('english'))
    cleaned_tokens = [w for w in tokens
        if w not in string.punctuation and w not in stop_words]
    return " ".join(cleaned_tokens)
cleaned_corpus = [preprocess_text(d) for d in corpus]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(cleaned_corpus)
print("Vocabulary:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", tfidf_matrix.toarray())
```

#4B CFG parsing (ChartParser)

```
import nltk
grammar = nltk.CFG.fromstring("""
  S -> NP VP
  NP -> 'describe'
  VP -> 'your' 'work'
""")
parser = nltk.ChartParser(grammar)
sentence = "describe your work"
tokens = sentence.split()
trees = list(parser.parse(tokens))
for tree in trees:
    print(tree)
if len(trees) > 0:
    print("\nResult: The sentence is grammatically valid.")
else:
    print("\nResult: The sentence does not match the grammar.")
```

#5A N-grams

```
import nltk
from nltk.util import ngrams
nltk.download('punkt')
text = "This is a simple sample text to test ngrams."
tokens = nltk.word_tokenize(text)
for i in range(1, 4):
    print(f"\n--- {i}-grams ---")
    for ngram in ngrams(sequence=tokens, n=i):
        print(ngram)
```

#5B Bigram probability

```
from collections import Counter
corpus = ["this is my cat", "this is my dog"]
words = " ".join(corpus).split()
uni = Counter(words)
bi = Counter(zip(words, words[1:]))
test = "this is my cat".split()
p = 1
for w1, w2 in zip(test, test[1:]):
    p *= bi[(w1, w2)] / uni[w1] if bi[(w1, w2)] else 0
print("Sentence:", " ".join(test))
print("Probability:", p)
# Expected: 0.5 ; P(w2|w1) = count(w1,w2)/count(w1)
```

#6 NER with Stanza

```
import stanza
stanza.download('en')
nlp = stanza.Pipeline('en', processors='tokenize,ner')
def extract_entities(text):
    doc = nlp(text)
    for e in doc.ents:
        print(e.text, "->", e.type)
documents = [
    "Barack Obama was born in Hawaii.",
    "Microsoft was founded by Bill Gates.",
    "Apple is located in California.",
    "John visited Bengaluru on Monday."
]
for i, text in enumerate(documents, 1):
    print("\nDocument", i)
    extract_entities(text)
```

#7 Stopword freq & TF-IDF (Reuters)

```
import nltk
from collections import Counter
from nltk.corpus import reuters, stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
nltk.download('reuters')
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
def analyze():
    docs = [reuters.raw(f) for f in reuters.fileids()]
    sw = set(stopwords.words('english'))
    count = Counter()
    for d in docs:
        count.update(w for w in nltk.word_tokenize(d.lower())
                     if w in sw)
    print("Stopwords:", count.most_common(10))
    v = TfidfVectorizer(stop_words='english')
    x = v.fit_transform(docs)
    words = v.get_feature_names_out()
    scores = x[0].toarray()[0]
    print("Top TF-IDF words:")
    for i in scores.argsort()[-10:][::-1]:
        print(words[i], round(scores[i], 3))
analyze()
```

#8 Sentiment analysis (Naive Bayes)

```
import nltk
from nltk.corpus import movie_reviews
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
nltk.download('movie_reviews')
X = [movie_reviews.raw(f) for f in movie_reviews.fileids()]
y = [movie_reviews.categories(f)[0] for f in movie_reviews.fileids()]
model = make_pipeline(TfidfVectorizer(stop_words='english'),
                      MultinomialNB())
model.fit(X, y)
for text in ["This movie is excellent", "This movie is terrible"]:
    print(text, ":", model.predict([text])[0].upper())
```